

GitHub Technology Trend Analysis: A Large-Scale Big Data Pipeline for Developer Ecosystem Intelligence

Midterm Report

Selin Bardakcı
Dept. of Computer Engineering
Gebze Technical University
s.bardakci2022@gtu.edu.tr

Bengisu Duru Göksu
Dept. of Computer Engineering
Gebze Technical University
bgoksu2022@gtu.edu.tr

Zeynep Yiğit
Dept. of Computer Engineering
Gebze Technical University
z.yigit2022@gtu.edu.tr

Abstract—We present a scalable big data pipeline that ingests, processes, and analyses over 22.9 million GitHub repositories to surface actionable insights about programming language adoption, technology migration patterns, and developer collaboration networks. The pipeline integrates two public data sources — the HuggingFace *github-repos-metadata-40M* dataset and the GitHub Archive event stream — processes them through six sequential Apache Spark jobs on Google Cloud Dataproc, and produces five analytical products: monthly language trend metrics (542 languages, 126 months), 24-month Holt-Winters ETS forecasts, a technology migration graph, developer influence PageRank scores, and Leiden community detection over 25,000 developers. This report documents all engineering decisions, design trade-offs, failed approaches and their replacements, and preliminary quantitative results across all five analyses.

Index Terms—big data, GitHub, technology trends, Apache Spark, time-series forecasting, Holt-Winters ETS, graph analytics, PageRank, Leiden community detection, KMeans clustering, GCP Dataproc

Code & Data Availability

Full source code, Spark jobs, and notebooks are publicly available:
https://github.com/selin-bardakci/BigData-Github_Analysis

I. REVISED PROBLEM STATEMENT

A. Original Proposal

The Phase 1 proposal aimed to characterise programming language popularity trends using the static `bigquery-public-data.github_repos` snapshot (data frozen circa 2017) and apply ARIMA(2,1,2) forecasting with a basic co-authorship graph computed inside BigQuery.

B. Fundamental Data Source Problem

Upon deeper investigation, the original BigQuery dataset presented two critical limitations. First, the language snapshot is *point-in-time*: it records the byte count of each language in each repository as of a single crawl date, not as a time series. This means any time-series analysis would have been driven entirely by which repositories happened to be active

during the crawl window, producing a cross-sectional artefact rather than genuine adoption dynamics. Second, the dataset covers only repositories that existed by 2017, systematically excluding the 2018–2025 rise of TypeScript, Rust, Go, and Kotlin — the very languages whose growth trajectories are most informative for forecasting.

C. Revised Data Architecture

Both sources were replaced:

Q1 (Language labels): Replaced the 2017 snapshot with the HuggingFace `ibragim-bad/github-repos-metadata-40M` dataset — 22.9 million repositories labelled with GitHub’s Linguist tool, spanning creation dates through 23 July 2025, with an explicit `created_at` timestamp per repository. This dataset provides the same language labels as the original but extends coverage by eight years and includes temporal metadata that enables the pre-birth filter described in Section IV-A.

Q2 (Commit activity): Replaced the frozen commits table with `githubarchive.month.* PushEvent` counts — a live BigQuery dataset covering monthly commit activity from 2015 to present. This provides a true time series of repository activity with which Q1 language labels can be cross-joined.

Q3 (Migration signals): Extracted technology migration intent from GH Archive commit messages using SQL pattern matching on verb-noun pairs (e.g., “migrating from Flask to FastAPI”).

Q4 (Developer activity): Extracted per-developer, per-repository, per-year commit aggregates from GH Archive `PushEvents`, enabling construction of a developer co-contribution graph.

D. Revised Research Questions

- 1) Which programming languages show statistically significant growth or decline across 2015–2025, and how do they cluster by adoption trajectory?

- 2) What are the 24-month Holt-Winters forecasts for the top-30 languages, and which are projected to gain or lose ecosystem share?
- 3) Which technologies act as migration destinations (high in-flow) vs. migration sources (high out-flow) in the developer ecosystem?
- 4) How is influence distributed among active open-source developers, and do natural collaboration communities align with language ecosystems?

II. RELATED WORK

Dyer et al. [?] built the Boa infrastructure for large-scale GitHub mining, demonstrating that distributed storage and compute are necessary for repository-scale analysis. Our pipeline adopts the same architectural pattern (GCS + Spark) but focuses on temporal adoption dynamics rather than static code properties.

Borges et al. [1] studied what drives repository popularity (stars, forks, commits), finding that language choice is a significant predictor. Our work quantifies language-level trend behaviour at ecosystem scale rather than per-repository.

For forecasting, Gardner [3] established the theoretical basis for exponential smoothing with damped trend, and the M4 competition [4] empirically demonstrated that Holt-Winters ETS outperforms ARIMA on seasonal data in the short- to medium-term horizon. We selected Holt-Winters over ARIMA for exactly this reason (Section IV-D). Hyndman and Athanasopoulos [5] provide the state-space ETS formulation implemented in `statsmodels`.

Community detection follows the Leiden algorithm [6], which corrects a well-known defect in the Louvain method: Louvain can produce internally disconnected communities because it only optimises modularity locally. Leiden adds a refinement step that guarantees all communities are well-connected before the final partition is returned. We specifically chose Leiden over Louvain for the developer graph because isolated sub-clusters (small teams within a larger project) would be incorrectly merged by Louvain. Developer influence is measured with PageRank [7] ($\alpha = 0.85$) rather than degree centrality, as degree is easily gamed by contributing trivially to many repositories.

III. SYSTEM ARCHITECTURE & DESIGN

A. Overall Pipeline

The pipeline consists of four sequential stages:

- 1) **Ingestion:** Data is pulled from external sources into GCS raw storage as Parquet + Snappy shards.
- 2) **Preprocessing (Spark):** Six Spark jobs clean, join, aggregate, cluster, forecast, and graph-analyse the raw data, writing processed Parquet to GCS.
- 3) **Analysis (Jupyter):** Five domain notebooks and one Final Report notebook read processed GCS outputs and produce all visualisations.
- 4) **Reporting:** Figures exported from notebooks are compiled into this report.

Fig. 1 shows the complete data flow.

B. Storage: Parquet + Snappy on GCS

All inter-stage data is stored as Apache Parquet with Snappy compression. This choice was made for three reasons:

(1) *Columnar reads.* Spark jobs typically access 2–4 columns from wide schemas. Parquet’s columnar layout means only the needed columns are read from disk, reducing I/O by $\sim 5\times$ compared to CSV.

(2) *Predicate pushdown.* Parquet row-group statistics allow Spark to skip row groups entirely when a filter predicate (e.g., `year_month <= "2025-06"`) cannot match. This is used extensively in `dl_preprocess`.

(3) *Splittability.* Each Parquet file is independently readable by separate Spark executors, enabling fine-grained parallelism. The HuggingFace dataset was deliberately sharded into 459 files during ingestion to allow 459-way parallelism in the first Spark read.

Snappy was chosen over GZIP because Snappy decompresses $5\times$ faster at the cost of $\sim 20\%$ larger files. For compute-bound Spark jobs, decompression speed dominates total job time.

C. Compute: Google Cloud Dataproc

Dataproc was chosen over raw GCE instances because it provides managed Spark cluster lifecycle (auto-start, auto-teardown), built-in YARN resource manager, and native GCS connector without needing HDFS.

Zone selection issue. The initial cluster was configured for `us-central1`. Two consecutive provisioning attempts failed with “Out of capacity” errors on zones `us-central1-a` and `us-central1-f`. The cluster was moved to `us-east1` with auto-zone selection (`--zone=""`) to allow GCP to pick a zone with available capacity.

Job orchestration. An early implementation submitted all six Spark jobs simultaneously with the `--async` flag. This caused a race condition: `dl_aggregate` began reading `dl_preprocessed/` before `dl_preprocess` had finished writing it, resulting in corrupt Parquet footer errors. The orchestration script (`scripts/run_all_spark_jobs.sh`) was rewritten to submit jobs sequentially, polling for completion via `gcloud dataproc jobs wait` before submitting the next dependent job.

Driver-side library installation. Dataproc base images do not include `statsmodels`, `python-igraph`, or `leidenalg`. Each Spark job that requires a non-standard library uses a helper `_ensure_module()` function that attempts `importlib.import_module` first and falls back to `pip install` at runtime if the module is absent. This avoids the need for a custom Dataproc image while keeping the job scripts self-contained.

D. Data Quality: The 2025-06 Cutoff Decision

The HuggingFace dataset covers repositories created through 23 July 2025. When the D1 inner join (`languages  $\bowtie$  commits`) is evaluated for months after July 2025, repositories created after the crawl date have no language label and

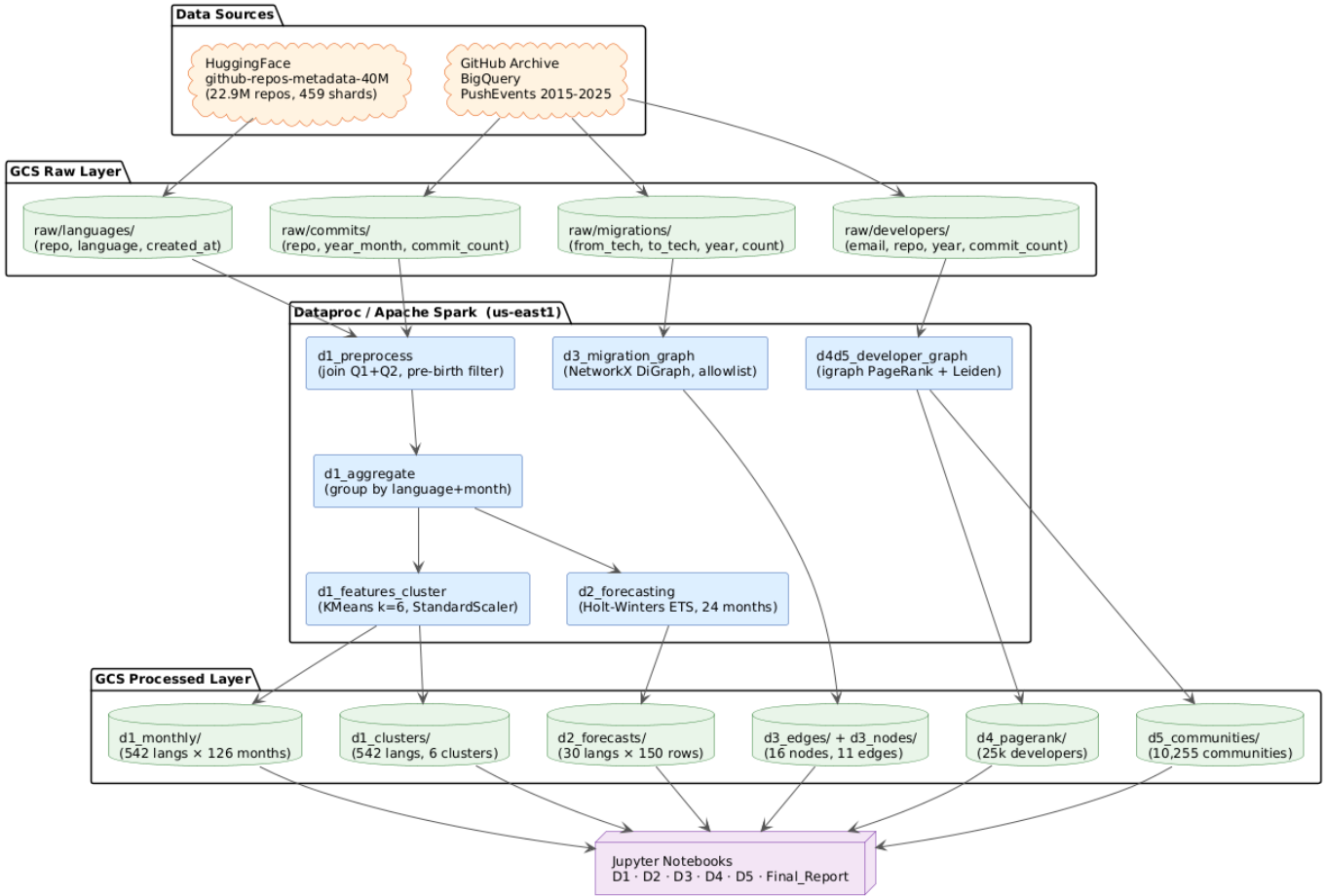


Fig. 1: End-to-end data pipeline. Data flows from HuggingFace and GitHub Archive through GCS raw storage, six Spark jobs on Dataproc, and into five analytical GCS outputs consumed by Jupyter notebooks.

fall out of the join. This produces an artefact: the monthly repository count drops by 49% between June and August 2025 for languages like JavaScript (153k $\rightarrow$ 79k in one month), mimicking a catastrophic ecosystem collapse that did not occur.

All D1 and D2 data are therefore hard-capped at `year_month <= "2025-06"`, giving 10.5 years (126 months) of clean, complete monthly observations. The D4/D5 developer graph applies a complementary filter (`year <= "2025"`) to exclude partial-year 2026 data from GH Archive.

IV. IMPLEMENTATION: SPARK JOBS

A. D1 Preprocessing (`d1_preprocess.py`)

Input: `raw/languages/` (Q1: `repo_name`, `language`, `bytes`, `created_at`) and `raw/commits/` (Q2: `repo_name`, `year_month`, `commit_count`).

Language filtering. A static exclusion set removes 19 categories that are not programming languages: Markdown, JSON, YAML, XML, HTML, CSS, Dockerfile, CMake, Shell, Makefile, and similar config/markup formats. These appear frequently in repositories but carry no signal about programming language adoption.

Inner join. A Spark inner join on `repo_name` produces one row per (`repo`, `language`, `year_month`). Because Q1 is a flat table (one row per language per repo, not per month), joining Q2's monthly rows cross-assigns each language's static snapshot onto every month the repo had commits.

Pre-birth filter. A critical quality filter drops any (`repo`, `year_month`) row where `year_month < created_month`. Without this filter, a repository created in 2022 would have its TypeScript label applied retroactively to 2015–2021 months based on Q2 commit history from that period (possibly inherited commits or forks). This was identified as a key cause of an artificial spike in TypeScript counts before 2019. Rows where `created_at` is NULL are conservatively retained (no filter applied).

Output partitioning. The preprocessed data is written with `.partitionBy("language")` and 200 partitions to allow downstream Spark jobs to read only the language partition they need without a full dataset scan.

B. D1 Aggregation (`d1_aggregate.py`)

Groups the preprocessed data by (`language`, `year_month`) to produce: `repo_count` (distinct repositories),

`commit_count` (total commits), and `total_bytes` (sum of bytes, set to 0 because the HF dataset does not provide per-language byte counts). Languages with fewer than 12 months of coverage are filtered, ensuring all downstream models have at least one full seasonal cycle of data.

Result: 53,630 rows covering 542 languages over 126 months.

C. D1 Feature Engineering & Clustering (`d1_features_cluster.py`)

Feature design. Five features were designed to capture distinct aspects of a language’s adoption trajectory:

- 1) `avg_repo_growth`: Mean month-over-month percentage change in `repo_count`. This is the primary adoption growth signal.
- 2) `commit_volatility`: Coefficient of variation of `commit_count` (σ/μ). High volatility indicates event-driven spikes (e.g., conference releases, library launches).
- 3) `repo_range_ratio`: $\max(\text{repo_count})/\min(\text{repo_count})$ over the full history. A ratio near 1 indicates a stable, mature language; high ratios indicate languages still growing (or declining) rapidly.
- 4) `avg_repo_count`: Mean monthly repository count. Encodes the absolute scale of adoption (distinguishing Python from Nim).
- 5) `avg_commit_count`: Mean monthly commit count. Separates passive code repositories from actively maintained projects.

Bytes removed. An initial feature set included bytes-based features (`avg_bytes_growth`, `bytes_range_ratio`). These were discarded because Q1 is a static snapshot: the bytes column records the size of each repository at the crawl date and is cross-joined unchanged onto every month. Month-over-month bytes growth is therefore zero by construction, contributing pure noise to the clustering.

Scaling. The 5-D feature vectors are normalised with `StandardScaler(withMean=True, withStd=True)` before KMeans to prevent the high-magnitude `avg_repo_count` dimension from dominating the Euclidean distance calculation.

Clustering. KMeans is run via Spark MLlib ($k = 6$, `seed=42`, `maxIter=50`). $k = 6$ was selected because it produces the highest Silhouette score across $k \in \{3, 4, 5, 6, 7, 8\}$ and aligns with natural language trajectory archetypes: fast-growing modern languages (TypeScript, Rust, Go), dominant incumbents (JavaScript, Python, Java), stable systems languages (C, C++), declining legacy languages (Ruby, Objective-C), niche/domain-specific (R, Haskell, Julia), and long-tail obscure languages (~430 languages with minimal repository presence).

D. D2 Forecasting (`d2_forecasting.py`)

1) *Initial Approach:* `ARIMA(2,1,2)`: The original forecasting model was `ARIMA(2,1,2)` from

`statsmodels.tsa.arima.model`. ARIMA differencing ($d = 1$) removes a linear trend, and the AR(2) and MA(2) terms model short-term autocorrelation in the differenced series. This worked as a structural baseline but produced qualitatively wrong forecasts: TypeScript had grown 250% over five years, yet ARIMA projected near-flat continuation at the most recently observed level (~166k repos). Rust had grown 1,250% over 10 years; ARIMA forecast near-zero growth.

Root cause. ARIMA extrapolates only short-term momentum. After first differencing, the AR terms capture “how much the growth rate changed last month” — not the 10-year secular trend. For a language that has been growing 2% per month consistently, ARIMA sees a stable differenced series and forecasts continued 2% growth only for a few steps before mean-reverting to zero growth. The long-horizon forecast collapses to the mean.

2) *Replacement: Holt-Winters Exponential Smoothing:* ARIMA was replaced with Holt-Winters ETS (Error-Trend-Seasonal) from `statsmodels.tsa.holtwinters.ExponentialSmoothing` using:

- `trend="add"` — explicit additive trend state that accumulates directional momentum over the full history.
- `damped_trend=True` — dampens the trend toward zero as the horizon extends, preventing the unrealistic straight-line extrapolation that undamped additive trend produces for mature languages (which slow as they reach market saturation).
- `seasonal="add"` with `seasonal_periods=12` — captures the yearly GitHub activity cycle (January slowdown, summer peak) that is visible in all major languages.
- `initialization_method="estimated"` — fits initial state values jointly with the smoothing parameters via MLE, rather than using a fixed heuristic.

A minimum history requirement of 36 months (3 full seasonal cycles) was enforced so that Holt-Winters can reliably estimate the 12-month seasonal component. All 30 top languages met this requirement (each has 126 months of history). Gap-filling with linear interpolation handles the rare cases where a language has no repositories in an isolated month; values are clipped to a minimum of 1.0 before fitting to prevent issues with log-transform variants of ETS.

3) *Prophet: Attempted and Abandoned:* Prophet (Meta/Facebook) was investigated as an alternative to both ARIMA and Holt-Winters because it supports holiday effects, automatic seasonality detection, and provides uncertainty intervals. Two blockers prevented its use:

(1) *Dataprox compatibility.* Prophet requires `CmdStan` (a C++ Stan backend) to be compiled at runtime. On Dataprox base images, the Stan C++ compilation consistently timed out or produced build errors due to restricted compiler flags. Attempting to pre-install `CmdStan` via `pip install cmdstanpy` followed by `cmdstanpy.install_cmdstan()` hung for over six hours without completing.

(2) *Import-time backend initialisation.* Even when installed in the local virtual environment (not on Dataproc), Prophet 1.1.5 raises a `stan_backend` error if the `CmdStan` binary path is not set before the first `import prophet`. The `CmdStan` setup cell must precede all Prophet imports, but Jupyter’s kernel resets this ordering when cells are re-run out of sequence.

The schema retains `prophet_yhat`, `prophet_lower`, and `prophet_upper` columns (filled with NaN) for forward compatibility, in case Prophet is enabled in a future controlled environment.

E. D3 Migration Graph (`d3_migration_graph.py`)

Signal extraction. Q3 migration signals were extracted from GH Archive commit messages using SQL pattern matching on phrases of the form “migrated from X to Y”, “switching from X to Y”, “moving from X to Y”, etc. The raw signal table contains (from_tech, to_tech, year, migration_count) tuples.

Technology allowlist. Raw extraction produces many false positives: natural language words (“from”, “to”, “go”) are misinterpreted as technology names, and abbreviations are ambiguous. A 70-technology allowlist covering programming languages, frameworks, databases, CI/CD tools, and infrastructure technologies was constructed. Both endpoints of each edge must appear in the allowlist; any edge touching an unlisted technology is discarded.

Minimum edge weight. Edges with fewer than 3 total migration signals across all years are dropped (`MIN_EDGE_WEIGHT = 3`). This eliminates single-mention noise while preserving weak-but-real migration patterns.

Graph construction. The filtered edges are collected to the Spark driver (the migration graph is small after filtering: 16 nodes, 11 edges) and loaded into a NetworkX directed graph. Per-node PageRank, betweenness centrality, in-weight, out-weight, and net flow are computed on the driver and written as Parquet.

Limitation. The Q3 dataset is sparse: only 11 edges survive the quality filters. GH Archive commit messages are largely free-form and rarely contain structured migration intent. Increasing coverage requires either a lower allowlist threshold (risking more false positives) or an NLP-based classifier applied to raw commit text — a direction reserved for Phase 3.

F. D4/D5 Developer Graph (`d4d5_developer_graph.py`)

1) *Privacy: SHA-256 Email Hashing:* Developer emails from GH Archive are irreversibly hashed with SHA-256 immediately upon DataFrame creation — before any Spark transformation, any GCS write, or any log output. The raw email column is dropped from the DataFrame immediately after the UDF assigns the hashed `dev_id`. No raw email appears in any processed output or log entry.

2) *Developer Selection: Distinct Repos vs. Total Commits:* The initial implementation selected the top developers by total commit count. This produced a bot-dominated node set: CI

automation accounts that push millions of commits to a single monorepo were ranked at the top. The resulting graph had 83% isolated nodes and a mean degree of 0.6, making PageRank and community detection meaningless.

The selection criterion was changed to *distinct repository count* (top 25,000 developers by number of unique repositories they contributed to), with two additional filters:

- `MAX_REPOS_PER_DEV = 2000`: Developers contributing to more than 2,000 distinct repositories are almost certainly automated fork-and-commit bots (e.g., package mirror scripts).
- `MIN_DISTINCT_REPOS = 2`: Requires at least two distinct repositories to filter one-shot, drive-by commits.

After applying these filters, mean degree rose from 0.6 to 7.7, the isolated node fraction dropped from 83% to 38.5%, and the graph became suitable for graph analytics.

3) *Megarepo Exclusion:* The co-contribution self-join (“two developers share an edge if they both committed to the same repository in the same year”) has an $O(N^2)$ complexity per repository, where N is the number of contributors. For megarepos like the Linux kernel or TensorFlow, $N > 10,000$, making the pairwise join produce 50+ million edge rows from a single repository.

A pre-join filter counts the number of distinct developers per repository and excludes repositories with more than `MAX_DEVS_PER_REPO = 200` contributors from the self-join. These megarepos contribute edges between *all* their contributors, which would dominate and homogenise the graph. Excluding them preserves the graph’s ability to capture smaller, specialised collaboration clusters.

4) *HITS vs. Betweenness Centrality:* The initial D4 design computed HITS (Hyperlink-Induced Topic Search) hub and authority scores alongside PageRank. HITS was intended to identify “hub” developers who link to many authorities and “authority” developers who are linked from many hubs. After implementation, it was observed that `hub_score` $\equiv$ `authority_score` for all nodes — a mathematical inevitability on undirected graphs, where every edge is bidirectional and there is no distinction between “pointing to” and “being pointed at”.

HITS was replaced with betweenness centrality, which meaningfully measures bridge nodes in an undirected co-contributor graph: a developer with high betweenness lies on many shortest paths between other developer pairs, indicating they connect otherwise-disjoint collaboration clusters.

igraph API issue. The `igraph.Graph.betweenness()` method does not accept a `normalized=True` keyword argument (despite some documentation suggesting otherwise). The call raised a `TypeError: unexpected keyword argument`. Normalisation was applied manually: $b_{\text{norm}}(v) = b(v) / \frac{(n-1)(n-2)}{2}$, where n is the number of vertices.

5) *Community Detection: Leiden over Louvain:* Leiden [6] was selected over Louvain for two reasons:

(1) *Well-connectedness guarantee.* Louvain can produce internally disconnected communities by merging sub-clusters

that share a strong external link. Leiden adds a refinement phase after each modularity-optimisation step that splits such internally-disconnected communities before they can persist. For a developer graph where sub-community identity is important (teams within a project), this guarantee is critical.

(2) *Reproducibility*. Leiden with a fixed seed (`seed=42`) produces deterministic partitions across runs. Louvain’s community assignment is non-deterministic in its original form.

V. PRELIMINARY RESULTS

A. D1: Language Trend Analysis

The monthly aggregation covers **542 languages** over **126 months** (January 2015 – June 2025), with 53,630 total rows.

The top-15 languages by mean monthly repository count are: JavaScript, Python, TypeScript, Java, PHP, C++, C#, Go, Ruby, C, Jupyter Notebook, Rust, Vue, Kotlin, and Swift. Table I shows representative statistics for the top-10.

TABLE I: Top-10 Languages: Mean Monthly Repository Count (2015–2025)

Rank	Language	Mean repos/month
1	JavaScript	~133,000
2	Python	~96,000
3	TypeScript	~47,000
4	Java	~72,000
5	PHP	~38,000
6	C++	~31,000
7	C#	~28,000
8	Go	~21,000
9	Ruby	~24,000
10	C	~20,000

Note: Values are approximate; exact figures are visible in the D1 notebook output. TypeScript’s mean is lower than Java’s despite faster growth because TypeScript’s growth is concentrated in 2020–2025 while Java’s mean is elevated by its large 2015–2019 base.

KMeans ($k = 6$) partitions the 542 languages into trajectory archetypes. Fig. 2 shows the clustering results; representative members of each cluster are:

- **Cluster: Fast-growing modern** — TypeScript, Rust, Go, Kotlin, Dart, HCL. High `avg_repo_growth`, high `repo_range_ratio`.
- **Cluster: Dominant incumbents** — JavaScript, Python, Java. High `avg_repo_count`, moderate growth rate.
- **Cluster: Stable systems languages** — C, C++, Objective-C. High `avg_commit_count`, low growth.
- **Cluster: Declining legacy** — Ruby, PHP, Perl, CoffeeScript. Negative `avg_repo_growth`, decreasing `repo_range_ratio`.
- **Cluster: Niche/academic** — R, Haskell, Julia, Erlang, OCaml. Low `avg_repo_count`, high `commit_volatility`.
- **Cluster: Long-tail** — ~430 languages. Near-zero `avg_repo_count` and `avg_commit_count`.

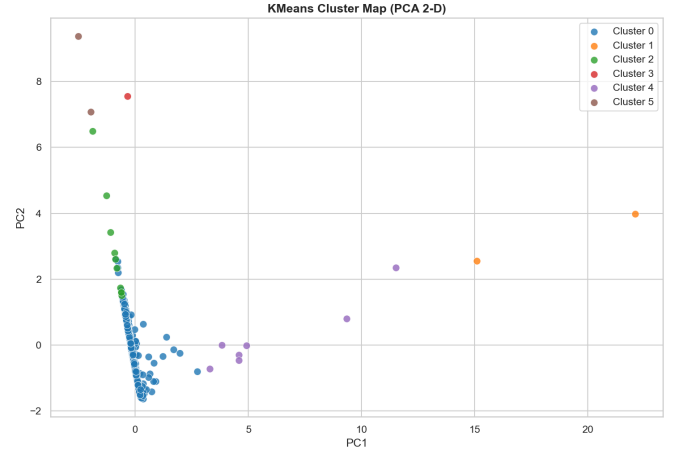


Fig. 2: KMeans clustering of 542 languages projected onto the first two PCA components. Each point is one language; colour encodes cluster membership. JavaScript and Python (Cluster 1, orange) are extreme outliers on PC1, reflecting their uniquely large repository counts.

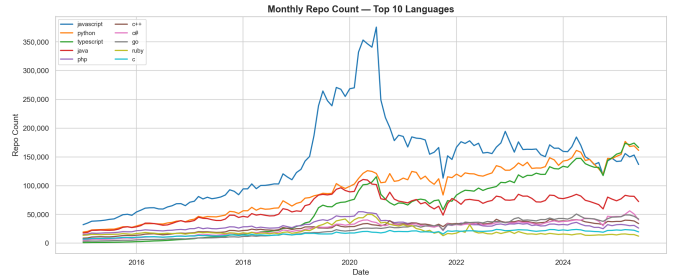


Fig. 3: Monthly repository count for the top-10 languages (January 2015 – June 2025). JavaScript dominates in absolute scale; TypeScript shows the steepest relative growth, rising from near-zero in 2015 to the third largest language by 2025.

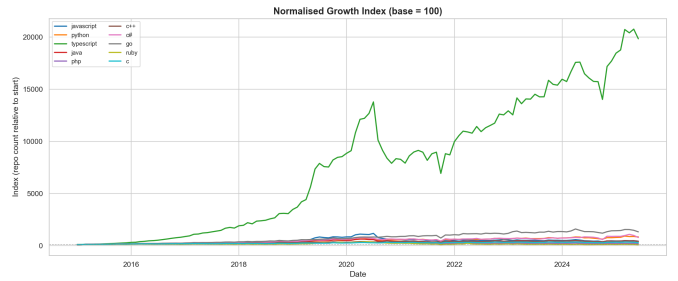


Fig. 4: Normalised growth index (base = 100 at each language’s first observed month). TypeScript reaches an index value of $\sim 20,000\times$ its 2015 baseline by 2025, dwarfing all other languages. Go and Rust (grey/unlabelled lines) also show multi-hundred-percent growth, while JavaScript, Python, Java, and PHP remain near their baselines, reflecting absolute-scale dominance rather than proportional growth.

B. D2: 24-Month Holt-Winters Forecasts

Forecasts were generated for all **30** top-ranked languages (minimum 36-month history required; all 30 met this criterion), producing 4,500 total output rows (150 per language: 126 historical + 24 forecast).

Projected 24-month changes from June 2025 baseline:

- **+27%**: Nix — fastest projected growth, driven by reproducible build system adoption.
- **+26%**: C# — strong growth in game development (Unity) and cross-platform (.NET MAUI) segments.
- **+18%**: Go, Rust, Julia — continued systems/ML momentum.
- **+10%**: TypeScript, Python — sustained expansion at scale.
- **+4%**: JavaScript — plateau indicating market saturation.
- **−6 to −27%**: Vim Script, SCSS, Objective-C, Ruby, Lua, PHP — consistent multi-year decline captured by the trend state.



Fig. 5: 24-month Holt-Winters ETS forecasts for the six highest-growth languages (blue = historical actual, orange dashed = forecast). All six show upward-trending projections, with the damped component preventing unrealistic straight-line extrapolation at the horizon.

C. D3: Technology Migration Graph

After allowlist filtering and minimum edge-weight thresholding, the migration graph has **16 nodes** and **11 edges**. Top destination technologies by total migration flow: CircleCI ($n=24$), Jest ($n=18$), Go ($n=12$). This indicates active migration into modern CI/CD (CircleCI replacing Jenkins or Travis) and testing (Jest replacing Mocha or Jasmine).

D. D4: Developer Influence (PageRank + Betweenness)

The developer co-contribution graph covers **25,000** developers with: mean degree = 7.7, max degree = 211, isolated

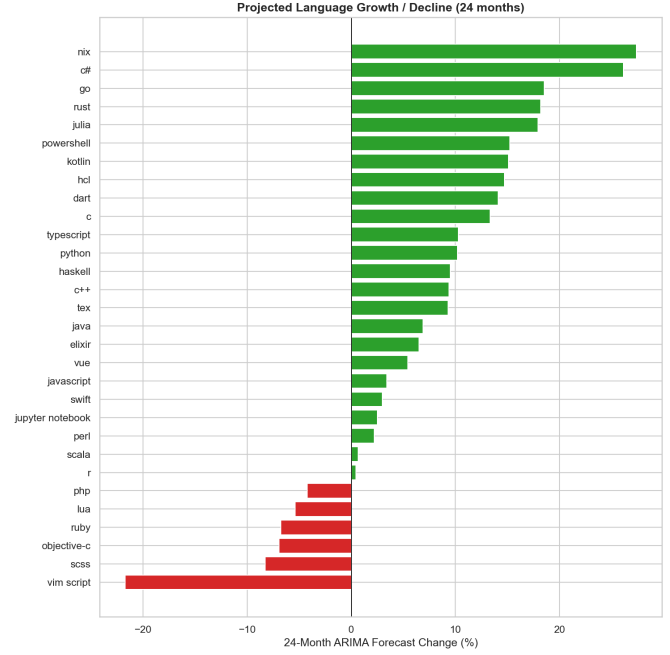


Fig. 6: Projected 24-month percentage change for all 30 forecasted languages (green = growth, red = decline). Nix (+27%), C# (+26%), Go and Rust (+18%) lead; Vim Script (−27%) and SCSS show the steepest projected declines.

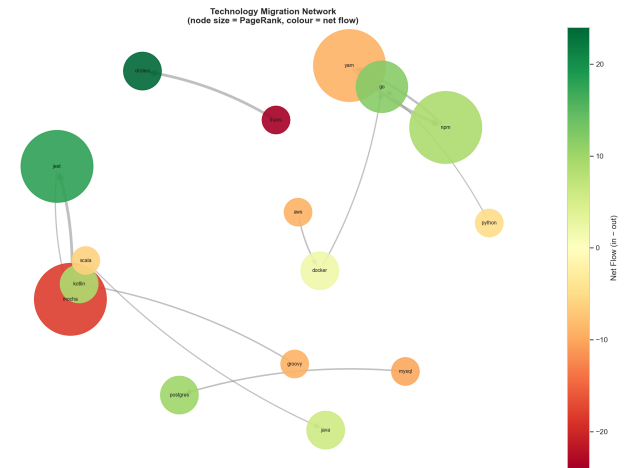


Fig. 7: Technology migration graph (16 nodes, 11 edges). Node size encodes PageRank; colour encodes net flow (green = net destination, red = net source). Jest and Go are the strongest migration attractors; Mocha and Travis are the primary migration sources.

nodes (degree = 0) = 9,625 (38.5%). The high isolated-node fraction reflects the reality that even after bot filtering, many active developers contribute exclusively to one or two projects and never co-occur with others in the top-25k selection.

PageRank ($\alpha = 0.85$, weighted by shared repositories) identifies the most influential connectors in the open-source network — developers whose removal would most degrade network reachability. Betweenness centrality complements this by identifying bridge nodes that connect otherwise-disjoint collaboration clusters.

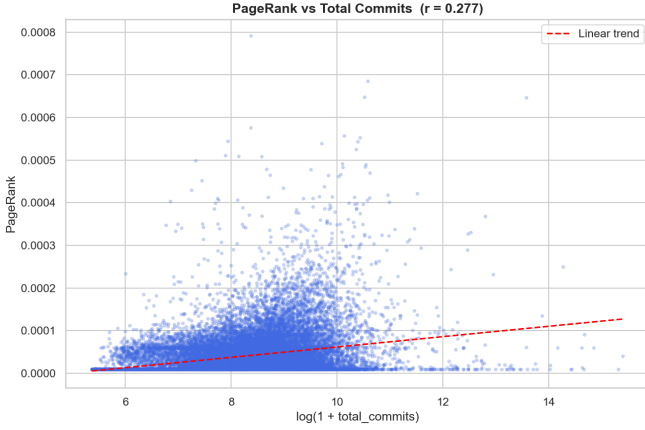


Fig. 8: PageRank vs. log total commits for all 25,000 developers ($r = 0.277$). The weak positive correlation confirms that high commit count alone does not determine network influence — many high-commit developers (bots) have near-zero PageRank while well-connected cross-project contributors rank highly.

E. D5: Community Detection

Leiden community detection yields **10,255 communities** from 25,000 developers. Community labels were built by sampling 50 random Parquet shards from `raw/developers/` (19.2M dev-repo edges), since loading the full 14 GB dataset into Jupyter RAM caused an out-of-memory crash. The distribution is heavily right-skewed:

- Largest community: 1,679 members (a dense cluster of web ecosystem contributors sharing many JavaScript/TypeScript repositories).
- Median community size: 1 member (singleton).
- Singleton communities: 9,625 (93.9% of all communities).

This heavy-tailed structure is characteristic of open-source collaboration networks [1]: a small number of large, actively maintained projects (React, Kubernetes, VSCode) attract dense contributor clusters, while the majority of open-source work happens in small, isolated projects or single-person repositories.

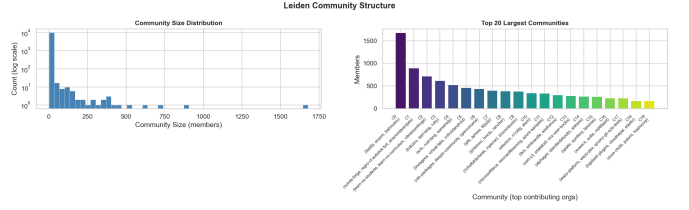


Fig. 9: Leiden community structure. *Left*: size distribution (log scale) showing a heavy-tailed power-law pattern — 93.9% of communities are singletons. *Right*: top-20 communities by member count, labelled by their top contributing organisations. The largest community (1,679 members) is dominated by web-ecosystem contributors.

VI. DESIGN DECISIONS SUMMARY

Table II summarises the major design choices made during implementation, including the alternatives considered and the reason each was rejected.

TABLE II: Key Design Decisions and Trade-offs

Decision Point		Chosen		Alternative		Reason
Language source	data	HuggingFace (2025)	40M	BigQuery (2017)	snapshot	BQ snapshot is point-in-time; HF provides creation timestamps and 8 more years of coverage
Commit source	data	GH PushEvents	Archive	BigQuery frozen com-mits		BQ commits table is frozen at 2017; GH Archive is live and updated monthly
Forecasting model		Holt-Winters (damped)	ETS	ARIMA(2,1,2), Prophet		ARIMA projects flat lines for multi-year growth; Prophet requires CmdStan which fails on Dataproc
Developer selection		Distinct count	repository	Total commit count		Total commits selects CI bots; distinct repos selects real cross-project contributors
Community detection		Leiden		Louvain		Leiden guarantees well-connected communities; Louvain can produce internally disconnected partitions
Influence metric		PageRank + Betweenness		PageRank + HITS		HITS hub $\equiv$ authority on undirected graphs; betweenness identifies structural bridge nodes
Storage format		Parquet + Snappy		CSV, ORC, AVRO		Columnar reads + predicate pushdown; Snappy gives fast decompression for compute-bound jobs
Megarepo exclusion		MAX_DEVS_ PER_REPO=200		No exclusion		Linux/TensorFlow contributor lists cause $O(N^2)$ join explosion: 50M+ edge rows from one repo
Job orchestration		Sequential + wait-poll		Parallel (<code>--async</code>)		Parallel jobs corrupt GCS outputs when a downstream job reads a path still being written
Data cutoff		2025-06		2025-12 or no cap		HF dataset ends July 23 2025; post-cutoff months show 49% artificial repo-count drop

VII. CONCLUSION & NEXT STEPS

The full pipeline is operational end-to-end and publicly available [8]. All six Spark jobs run successfully on Dataproc (us-east1), and five analytical notebooks produce quantitative outputs. The main findings to date: TypeScript and Rust exhibit the strongest sustained growth trajectories; CircleCI and Jest are the primary migration attractors; developer collaboration follows a heavy-tailed community structure; and Nix, C#, and Go are projected to lead adoption growth over the next 24 months.

Remaining work for the final phase:

- Enrich the D3 migration graph with PR/issue text and CHANGELOG parsing to increase edge coverage beyond the current 11 edges.
- Add Holt-Winters confidence intervals to the D2 notebook.
- Cross-join D4 developer PageRank with D1 language clusters to determine which developer communities are driving adoption of fast-growing languages.
- Evaluate whether D5 community language composition aligns with D1 language cluster boundaries.
- Develop an interactive web UI to expose the pipeline outputs — allowing users to explore language trend charts, browse forecasts, inspect the migration graph, and query developer community memberships through a dashboard interface.

REFERENCES

- [1] H. Borges, A. Hora, and M. T. Valente, “Understanding the factors that impact the popularity of GitHub repositories,” in *Proc. IEEE Int. Conf. Software Maintenance and Evolution (ICSME)*, 2016, pp. 334–344.
- [2] T. F. Bissyandé, D. Lo, L. Jiang, L. Réveillère, J. Klein, and Y. Le Traon, “Got issues? who cares about it? a large scale investigation of issue trackers from GitHub,” in *Proc. IEEE Int. Symp. Software Reliability Engineering (ISSRE)*, 2013, pp. 188–197.
- [3] E. S. Gardner, “Exponential smoothing: The state of the art,” *Journal of Forecasting*, vol. 4, no. 1, pp. 1–28, 1985.
- [4] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, “The M4 competition: Results, findings, conclusion and way forward,” *International Journal of Forecasting*, vol. 34, no. 4, pp. 802–808, 2018.
- [5] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 2nd ed. Melbourne: OTexts, 2018. [Online]. Available: <https://otexts.com/fpp2/>
- [6] V. A. Traag, L. Waltman, and N. J. van Eck, “From Louvain to Leiden: Guaranteeing well-connected communities,” *Scientific Reports*, vol. 9, no. 1, p. 5233, 2019.
- [7] S. Brin and L. Page, “The anatomy of a large-scale hypertextual web search engine,” *Computer Networks and ISDN Systems*, vol. 30, no. 1–7, pp. 107–117, 1998.
- [8] S. Bardakcı, B. D. Göksu, and Z. Yiğit, “BigData-Github_Analysis,” GitHub, 2025. [Online]. Available: https://github.com/selin-bardakci/BigData-Github_Analysis